

MECHATRONICS AND IOT

ASSIGNMENT-5

Design and Development of Worker Health and Safety Monitoring System for Construction Industries

TEAM MEMBERS:

JEEVA T	24MER020
NITHEESH M	24MER036
SANJAY S	24MER054
SIVA BALAJI S	24MER058
SIVA S	24MER059

Design and Development of Worker Health and Safety Monitoring System for Construction Industries

1. Project Overview

The Worker Health and Safety Monitoring System is an IoT-based system designed to monitor important safety and environmental parameters of construction workers. Construction workers are exposed to several hazards such as excessive temperature, harmful gases, unsafe environmental conditions, falls, and accidents.

The proposed system uses sensors to continuously monitor worker and workplace conditions. An ESP8266 NodeMCU acts as the main controller and processes the sensor data. When an unsafe condition is detected, the system activates an alarm and provides a warning to the worker.

The system can also transmit the collected data through Wi-Fi to a cloud platform, allowing supervisors to remotely monitor worker safety conditions.

Main components

- Pulse Sensor
- ESP8266 Node MCU
- DHT11 Sensor
- MQ2 Gas Sensor
- Buzzer
- OLED Display
- GPS Module (NEO-6M)
- Breadboard

2. Project Statement

Construction sites are among the most hazardous working environments, where workers are exposed to various physical, environmental, and occupational risks on a daily basis. Workers may be exposed to excessive heat, harmful gases, dust, sudden falls, unsafe movements, and other dangerous conditions that can affect their health and safety. Environmental conditions such as high temperature and humidity can cause heat stress, fatigue, dehydration, and reduced concentration, increasing the possibility of accidents. The system continuously monitors selected worker and environmental safety parameters and compares the sensor readings with predefined safety limits. When a parameter exceeds its safe range, the system generates an immediate warning using a buzzer and visual indicators such as LEDs.

3. Proposed Solution

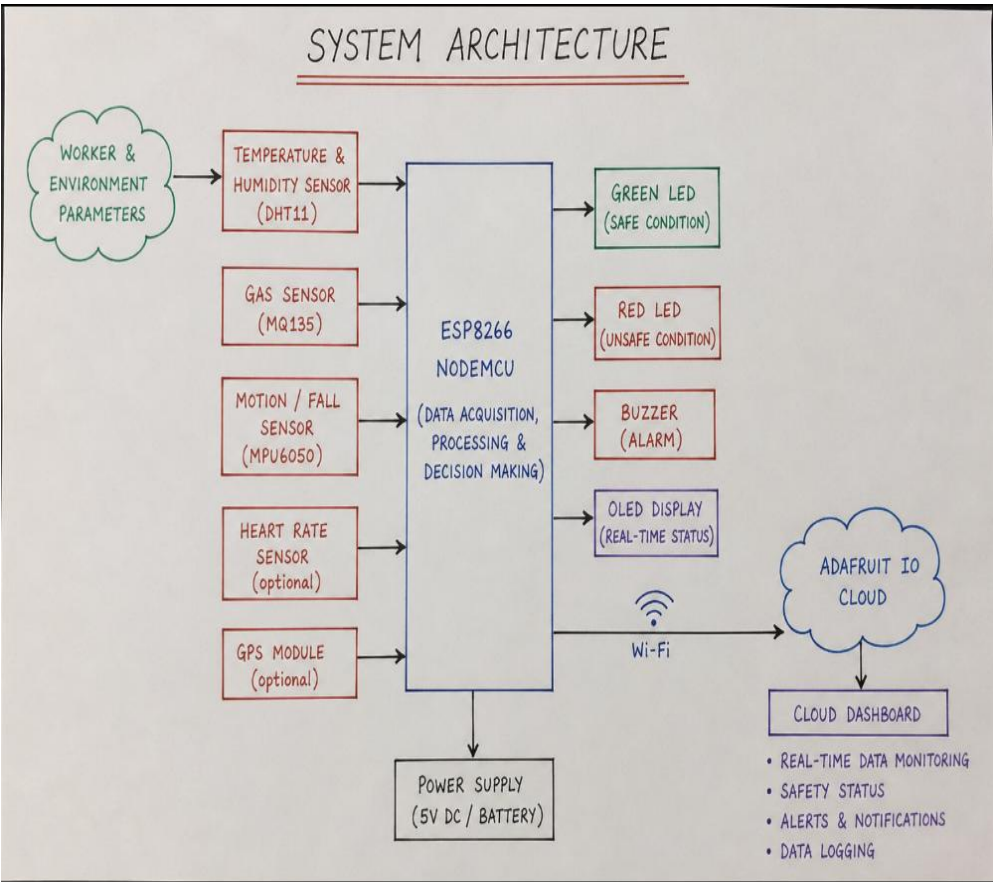
The proposed system uses multiple sensors connected to an ESP8266 NodeMCU.

The sensors continuously collect information about the worker's surrounding conditions and transmit it to the controller.

The system operates as follows:

1. Sensors monitor worker/environment parameters.
2. ESP8266 receives the sensor readings.
3. Sensor readings are compared with predefined safety limits.
4. If the conditions are normal, the system remains in safe mode.
5. If an unsafe condition is detected, an alarm is activated
6. Warning LEDs provide local indication.
7. Sensor data can be transmitted to a cloud platform through Wi-Fi.
8. Supervisors can remotely monitor the worker's safety condition

4. System Architecture:

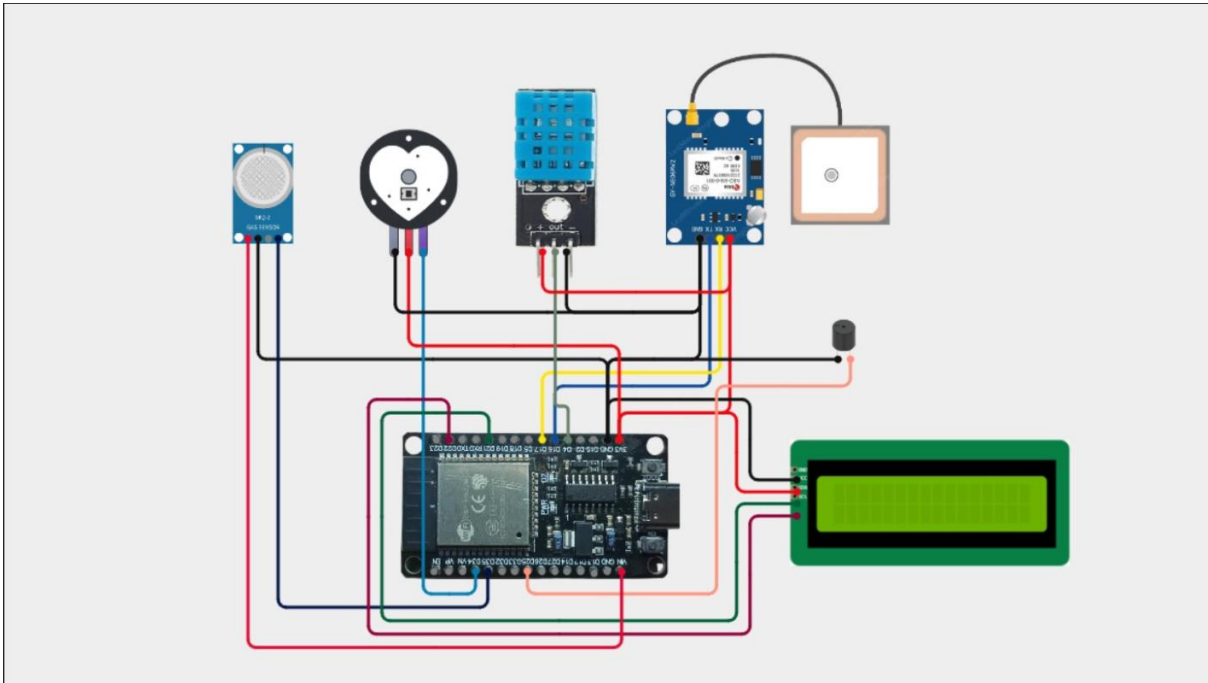


5. Circuit Table:

CIRCUIT TABLE

S. No.	Component	Pin	ESP8266 NodeMCU Pin	Description
1.	MQ135 Gas Sensor	VCC	VIN (5V)	Power Supply (5V)
		GND	GND	Ground
		AO	A0	Analog Output
2.	DHT11 Temperature & Humidity Sensor	VCC	3.3V	Power Supply (3.3V)
		GND	GND	Ground
		DATA	D2	Digital Data
3.	MPU6050 (Motion / Fall Detection Sensor)	VCC	3.3V	Power Supply (3.3V)
		GND	GND	Ground
		SCL	D1 (SCL)	I2C Clock
		SDA	D2 (SDA)	I2C Data
4.	Buzzer	+ (Positive)	D5	Buzzer Control
		- (Negative)	GND	Ground
5.	Green LED	Anode (+)	D6 through 220Ω	Green LED (Safe)
		Cathode (-)	GND	Ground
6.	Red LED	Anode (+)	D7 through 220Ω	Red LED (Unsafe)
		Cathode (-)	GND	Ground
7.	Power Supply	+5V	VIN (5V)	5V DC Supply
		GND	GND	Ground

6. Circuit Design:



7. Algorithm:

Algorithm for Air Pollution Monitoring System

Step 1: Start the system.

Step 2: Initialize the ESP8266 and all connected sensors.

Step 3: Connect the ESP8266 to the Wi-Fi network.

Step 4: Read temperature and humidity values.

Step 5: Read the gas sensor value.

Step 6: Read the motion/fall detection sensor.

Step 7: Compare the readings with predefined safety thresholds.

Step 8: If all values are within the safe range, turn ON the green LED and keep the buzzer OFF.

Step 9: If any parameter exceeds its safe limit, turn ON the red LED and activate the buzzer.

Step 10: Send the sensor data to the cloud platform.

Step 11: Continue monitoring.

Step 12: Repeat the process continuously.

8. Coding:

```
#include "AdafruitIO_WiFi.h"

#include <Wire.h>

#include <Adafruit_GFX.h>

#include <Adafruit_SSD1306.h>

#include <DHT.h>

#include <TinyGPSPlus.h>

#include <HardwareSerial.h>

// =====

// WIFI DETAILS

// =====

#define WIFI_SSID    "OnePlus Nord CE5 7C5C"

#define WIFI_PASSWORD "qkts5737"

// =====

// ADAFRUIT IO DETAILS

// =====

#define IO_USERNAME  "Sivasaravanan"

#define IO_KEY       "aio_ADVS50c7oG5Sx1R3FskFAq0fjUrB"

// Create Adafruit IO WiFi object

AdafruitIO_WiFi io(

  IO_USERNAME,

  IO_KEY,

  WIFI_SSID,

  WIFI_PASSWORD

);

// =====
```

```

// FEEDS
// =====

AdafruitIO_Feed *pulseFeed =
    io.feed("pulse");

AdafruitIO_Feed *temperatureFeed =
    io.feed("temperature");

AdafruitIO_Feed *humidityFeed =
    io.feed("humidity");

AdafruitIO_Feed *mq2Feed =
    io.feed("mq2");

AdafruitIO_Feed *latitudeFeed =
    io.feed("latitude");

AdafruitIO_Feed *longitudeFeed =
    io.feed("longitude");

AdafruitIO_Feed *satellitesFeed =
    io.feed("satellites");

AdafruitIO_Feed *gasAlertFeed =
    io.feed("gas-alert");

// =====

// DHT11
// =====

#define DHT_PIN 4

#define DHT_TYPE DHT11

DHT dht(DHT_PIN, DHT_TYPE);

// =====

// PULSE SENSOR
// =====

#define PULSE_PIN 34

// =====

```

```
// MQ2 GAS SENSOR
// =====
#define MQ2_PIN 35
// =====
// BUZZER
// =====
#define BUZZER_PIN 25
// =====
// OLED
// =====
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(
  SCREEN_WIDTH,
  SCREEN_HEIGHT,
  &Wire,
  -1
);
// =====
// GPS
// =====
#define GPS_RX 16
#define GPS_TX 17
HardwareSerial GPS(2);
TinyGPSPlus gps;
// =====
// VARIABLES
// =====
float temperature = 0;
```



```
float humidity = 0;
int pulseValue = 0;
int mq2Value = 0;
float latitude = 0;
float longitude = 0;
int satellites = 0;
bool gasAlert = false;

// =====
// TIMING
// =====

unsigned long lastSend = 0;
const unsigned long sendInterval =
    15000;
// =====

// SETUP
// =====

void setup()
{
    Serial.begin(115200);
    delay(1000);
    // -----
    // DHT
    // -----
    dht.begin();
    // -----
    // SENSOR PINS
    // -----
    pinMode(PULSE_PIN, INPUT);
    pinMode(MQ2_PIN, INPUT);
```

```
pinMode(BUZZER_PIN, OUTPUT);

digitalWrite(
  BUZZER_PIN,
  LOW
);
// -----
// OLED
// -----

Wire.begin(21, 22);

if (!display.begin(
  SSD1306_SWITCHCAPVCC,
  0x3C))
{
  Serial.println(
    "OLED not detected!"
  );
}
else
{
  display.clearDisplay();
  display.setTextColor(
    SSD1306_WHITE
  );
  display.setTextSize(1);
  display.setCursor(10, 10);
  display.println(
    "SMART MONITOR"
  );
  display.setCursor(10, 30);
```

```
display.println(
  "Starting..."
);
display.display();
}
// -----
// GPS
// -----
GPS.begin(
  9600,
  SERIAL_8N1,
  GPS_RX,
  GPS_TX
);
// -----
// CONNECT TO ADAFRUIT IO
// -----
Serial.println();
Serial.println(
  "Connecting to Adafruit IO..."
);
io.connect();
// Wait for connection
while (io.status() < AIO_CONNECTED)
{
  Serial.print(".");
  delay(500);
}
Serial.println();
```

```

Serial.println(
  "Connected to Adafruit IO!"
);
display.clearDisplay();
display.setTextSize(1);
display.setCursor(0, 10);
display.println(
  "Adafruit IO"
);
display.setCursor(0, 30);
display.println(
  "Connected!"
);
display.display();
delay(2000);
}

// =====
// LOOP
// =====

void loop()
{
  // Keep Adafruit IO connection alive
  io.run();
  // -----
  // READ GPS
  // -----
  while (GPS.available())
  {
    gps.encode(

```

```

    GPS.read()

);
}
if (gps.location.isValid())
{
    latitude =
        gps.location.lat();
    longitude =
        gps.location.lng();
}
if (gps.satellites.isValid())
{
    satellites =
        gps.satellites.value();
}
// -----
// SEND SENSOR DATA
// -----
if (
    millis() - lastSend >=
    sendInterval
)
{
    lastSend = millis();
    // =====
    // DHT11
    // =====
    temperature =
        dht.readTemperature();

```

```
humidity =
    dht.readHumidity();
if (isnan(temperature) ||
    isnan(humidity))
{
    Serial.println(
        "DHT11 reading failed!"
    );
}
else
{
    Serial.print(
        "Temperature: "
    );
    Serial.print(
        temperature
    );
    Serial.println(" C");
    Serial.print(
        "Humidity: "
    );
    Serial.print(
        humidity
    );
    Serial.println(" %");
}
// =====
// PULSE SENSOR
// =====
```

```
pulseValue =
  analogRead(PULSE_PIN);
Serial.print(
  "Pulse Sensor: "
);
Serial.println(
  pulseValue
)
// =====
// MQ2
// =====
mq2Value =
  analogRead(MQ2_PIN);
Serial.print(
  "MQ2: "
);
Serial.println(
  mq2Value
)
// =====
// GAS ALERT
// =====
// Adjust this threshold after testing
if (mq2Value > 2000)
{
  gasAlert = true;
  digitalWrite(
    BUZZER_PIN,
    HIGH
```

```
)  
}  
else  
    gasAlert = false;  
    digitalWrite(  
        BUZZER_PIN,  
        LOW  
    );  
}  
// =====  
// GPS SERIAL OUTPUT  
// =====  
Serial.print(  
    "Latitude: "  
);  
Serial.println(  
    latitude,  
    6  
);  
Serial.print(  
    "Longitude: "  
);  
Serial.println(  
    longitude,  
    6  
);  
Serial.print(  
    "Satellites: "  
)
```



```
Serial.println(
    satellites
);
// =====
// SEND TO ADAFRUIT IO
// =====
Serial.println();
Serial.println(
    "Sending data to Adafruit IO..."
);
if (!isnan(temperature))
{
    temperatureFeed->save(
        temperature
    );
}
if (!isnan(humidity))
{
    humidityFeed->save(
        humidity
    );
}
pulseFeed->save(
    pulseValue
);
mq2Feed->save(
    mq2Value
);
latitudeFeed->save(
```

```
latitude
);
longitudeFeed->save(
    longitude
);
satellitesFeed->save(
    satellites
);
gasAlertFeed->save(
    gasAlert ? 1 : 0
);
Serial.println(
    "Data sent successfully!"
);
// =====
// OLED
// =====
display.clearDisplay();
display.setTextColor(
    SSD1306_WHITE
);
display.setTextSize(1);
display.setCursor(0, 0)
display.print("T:");
display.print(
    temperature,
    1
);
display.print("C ")
```

```
display.print("H:")
display.print(
  humidity,
  0
);
display.println("%")
display.setCursor(0, 15)
display.print("Pulse: ")
display.println(
  pulseValue
);
display.setCursor(0, 30)
display.print("MQ2: ")
display.println(
  mq2Value
);
display.setCursor(0, 45)
if (gasAlert)
{
  display.println(
    "GAS ALERT!"
  );
}
else
{
  display.println(
    "Gas Normal"
  );
}
```

```
display.display();

Serial.println(

"=====

");
```

9. System Working :

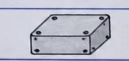
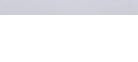


When the system is powered ON, the ESP8266 initializes all the connected sensors and output devices. The controller then establishes a Wi-Fi connection.

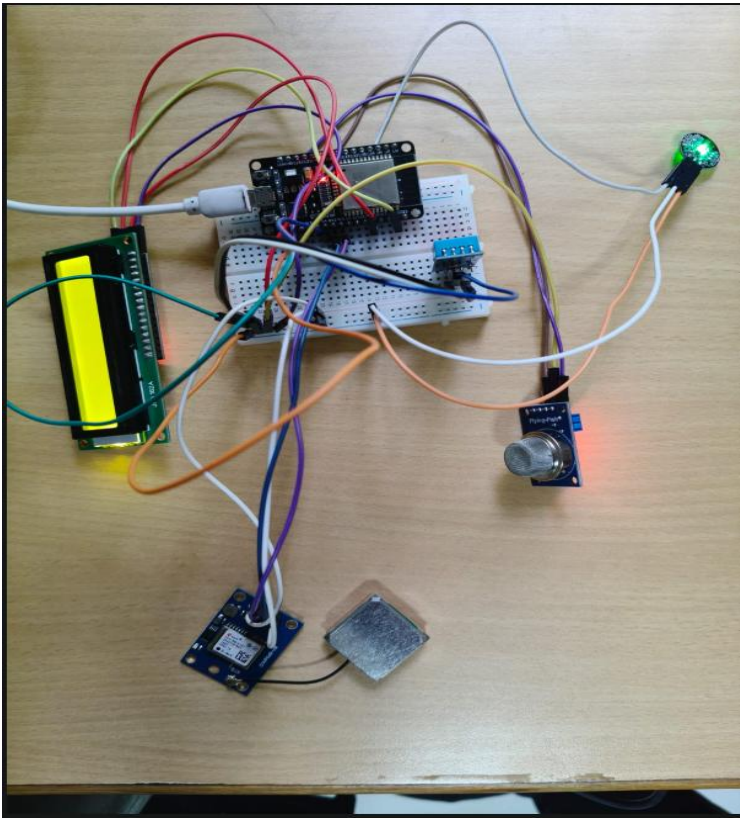
The temperature sensor measures the ambient temperature and humidity. The gas sensor detects potentially harmful gases. A motion or fall sensor can detect abnormal movement or a possible fall.

If an unsafe condition is detected, the red LED turns ON and the buzzer generates an audible warning. At the same time, the sensor information can be transmitted to a cloud platform for remote monitoring.

10. BILL OF MATERIAL:

BILL OF MATERIAL				
S. No.	COMPONENT	SPECIFICATION	QUANTITY	DESCRIPTION
1.	ESP8266 NodeMCU	ESP-12E	1	
2.	MQ135 Gas Sensor	5V DC	1	
3.	DHT11 Temperature & Humidity Sensor	3.3 – 5V DC	1	
4.	MPU6050 Motion / Fall Detection Sensor	3.3V DC	1	
5.	Buzzer	5V Active Buzzer	1	
6.	Green LED	3mm LED	1	
7.	Red LED	3mm LED	1	
8.	Resistors	220Ω , 1/4W	2	
9.	Breadboard	830 Tie-point	1	
10.	Jumper Wires	Male to Male	As required	
11.	Power Supply	5V DC (USB Adapter or Power Bank)	1	
12.	Project Box / Enclosure	Plastic Box	1	

11. Prototype Implementation:



The prototype is assembled using an ESP8266 NodeMCU as the main controller. The sensors are connected to the appropriate digital and analog input pins of the controller.

The temperature/humidity sensor, gas sensor, and motion/fall sensor continuously monitor the worker's surroundings.

Two LEDs and a buzzer are used for local alerts. The green LED indicates a safe condition, while the red LED and buzzer indicate an unsafe condition.

The ESP8266 can be connected to a Wi-Fi network to transmit the collected data to a cloud dashboard. This provides a method for remote monitoring by supervisors.

12. Results and Performance Analysis:

The developed **Worker Health and Safety Monitoring System** was successfully designed and implemented as an IoT-based safety monitoring prototype for construction industries. The system continuously monitors important environmental and worker-related parameters and provides an immediate warning when an unsafe condition is detected. The ESP8266 NodeMCU acts as the central processing unit and receives data from the connected sensors.

During testing, the sensors were operated under normal and simulated unsafe conditions to verify the response of the system. The sensor readings were continuously processed by the ESP8266 and compared with predefined safety threshold values. This allowed the system to distinguish between safe and potentially dangerous working conditions.

Under normal conditions, when the monitored parameters remained within the predefined safety limits, the system maintained the **safe condition**. The green LED was activated to indicate that the worker's environment was within the acceptable range, while the buzzer remained OFF.

This provided a simple visual indication to the worker that no immediate warning condition was detected.

When an unsafe condition was simulated, such as an increase in temperature, detection of harmful gas, or abnormal motion/fall condition, the ESP8266 identified the change in sensor readings. The system then activated the **red LED and buzzer**, providing an immediate local warning. This demonstrates that the system can respond automatically without requiring continuous manual observation by a supervisor.

The Wi-Fi communication capability of the ESP8266 also provides the possibility of transmitting the collected sensor data to a cloud platform. The data can be stored and displayed on a cloud dashboard, allowing supervisors or safety personnel to monitor the worker's condition remotely. This feature is particularly useful for large construction sites where workers may be located in different areas.

Performance Analysis

The prototype performed successfully in the following areas:

- **Sensor Monitoring:** The sensors continuously collect information about the worker's surrounding conditions. The ESP8266 successfully reads the sensor outputs and processes the received values.
- **Unsafe Condition Detection:** The system successfully identifies unsafe conditions by comparing sensor readings with predefined threshold values.
- **Alert Generation:** The combination of a red LED and buzzer provides an effective warning mechanism. The visual and audible alerts can help attract the worker's attention immediately when a dangerous condition occurs.
- **Response Time:** The system is designed to respond immediately after a sensor value crosses the programmed threshold. The response depends on the sensor sampling interval and processing delay. A shorter sampling interval can be used when faster detection is required.

- Remote Monitoring: The ESP8266 provides Wi-Fi connectivity, allowing sensor information to be transmitted to a cloud platform. This makes it possible for supervisors to observe safety parameters without being physically present near the worker.
- Continuous Operation: The system is capable of repeatedly reading sensor values, processing the data, checking safety conditions, and generating alerts. This makes it suitable for continuous safety-monitoring applications.
- Local Indication: The green and red LEDs provide an easy-to-understand indication of the current safety status. The green LED represents a normal condition, while the red LED represents an unsafe condition.

Overall, the developed system successfully demonstrates the ability to monitor worker safety parameters, identify unsafe conditions, generate local alerts, and support remote monitoring. The prototype can help improve safety awareness and reduce the time required to identify potentially dangerous conditions.